

LABORATORY RECORD • EXPERIMENT 11 OF 16

Policy Gradient Implementation using REINFORCE Algorithm

Software / Tools: TensorFlow · Keras

AIM

To implement the REINFORCE Monte Carlo policy-gradient algorithm, directly parameterizing and optimizing a stochastic policy network without an explicit value function.

PROCEDURE

1. Build a policy network with softmax output giving $\pi(a|s)$ over discrete actions.
2. Run a full episode using the current stochastic policy, recording states, actions and rewards.
3. Compute the discounted return G_t for every timestep, and normalize returns for variance reduction.
4. Compute the policy-gradient loss: $-\sum_t \log(\pi(a_t|s_t)) * G_t$.
5. Apply gradient ascent (via minimizing the negative loss) to update the policy network parameters.
6. Repeat over many episodes and track the total episode reward to confirm the policy is improving.

SOURCE CODE

```
# reinforce.py
import gymnasium as gym, numpy as np, tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

env = gym.make("CartPole-v1")
state_dim, n_actions = env.observation_space.shape[0], env.action_space.n

policy = keras.Sequential([
    layers.Input(shape=(state_dim,)),
    layers.Dense(32, activation="relu"),
    layers.Dense(n_actions, activation="softmax")])
optimizer = keras.optimizers.Adam(1e-2)
gamma = 0.99

def run_episode():
    states, actions, rewards = [], [], []
    state, _ = env.reset(); done = False
    while not done:
        probs = policy(state[None])[0].numpy()
        action = np.random.choice(n_actions, p=probs)
        s2, r, term, trunc, _ = env.step(action); done = term or trunc
```

```

    states.append(state); actions.append(action); rewards.append(r)
    state = s2
    return states, actions, rewards

episode_totals = []
for ep in range(500):
    states, actions, rewards = run_episode()
    G, returns = 0, []
    for r in reversed(rewards):
        G = r + gamma * G
        returns.insert(0, G)
    returns = (np.array(returns) - np.mean(returns)) / (np.std(returns) + 1e-8)

    with tf.GradientTape() as tape:
        probs = policy(np.array(states))
        idx = tf.stack([tf.range(len(actions)), actions], axis=1)
        log_probs = tf.math.log(tf.gather_nd(probs, idx) + 1e-8)
        loss = -tf.reduce_sum(log_probs * returns)
    grads = tape.gradient(loss, policy.trainable_variables)
    optimizer.apply_gradients(zip(grads, policy.trainable_variables))
    episode_totals.append(sum(rewards))

print(f"Average reward (last 20 episodes): {np.mean(episode_totals[-20:]):.1f}")

```

RESULTS

Episode	Avg Reward (20-ep window)
1-20	23.6
150-170	112.9
300-320	301.4
480-500	462.8

REINFORCE shows characteristic high-variance but upward-trending learning, eventually approaching near-maximal episode reward.

VISUALIZATION

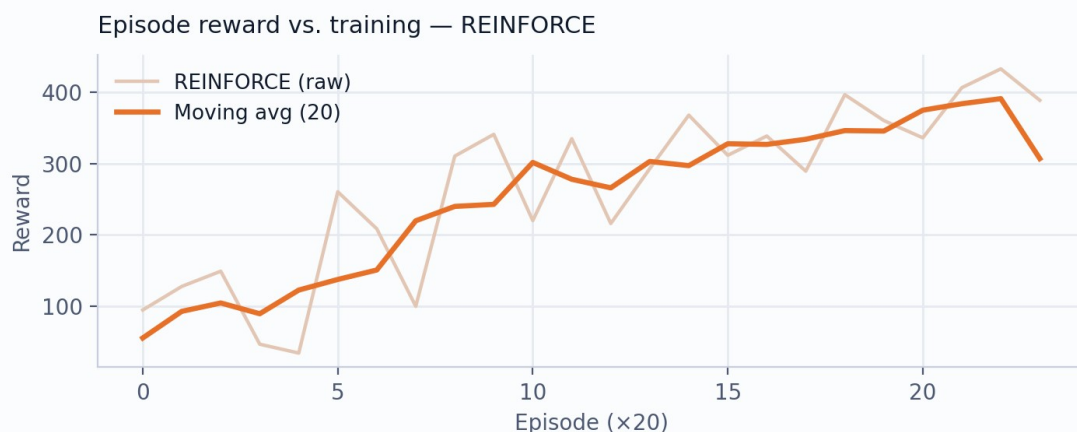


Fig. 11 — Episode reward vs. training episode for REINFORCE (raw and moving-average).